

Two pointers and linked lists

Your completed Day 2 tutorial, expanded for revision. Read the explanation, trace the example by hand, then close the notes and write the function yourself. Python indentation is preserved in every code block: it determines which statements belong to a loop or condition.

What does a pointer mean here?

For an array problem, a pointer is an integer index. For a linked-list problem, it is a reference to a node object. Both let us track a position, but their syntax differs.

Pattern	Meaning	Examples / pages
Opposite ends	left and right move inward	Palindrome (2), pair sum (3), reversal (4)
Read and write	read scans; write marks the next output position	Move Zeroes (5), remove duplicates (6)
Node references	Follow the next connection	Build nodes (7), traversal (8)
Slow and fast	Move one and two connections per iteration	Middle (9), cycle detection (10)

Basic syntax to keep beside you

```
nums = [10, 20, 30, 40]
left = 0
right = len(nums) - 1 # Last index: 3
print(nums[left])     # 10
left += 1              # Same as left = left + 1
right -= 1             # Same as right = right - 1
```

= assigns a value; == compares values; != checks whether values differ. A colon begins an indented block. **return** ends a function immediately and sends a result to its caller.

Complexity convention: n is the number of characters, list elements, or distinct reachable nodes. These algorithms use $O(n)$ time and $O(1)$ auxiliary space under the usual interview assumptions. Input storage and example-printing output are not included in auxiliary space.

1. Check an exact palindrome

A palindrome reads the same in both directions. Compare the first and last characters, then the next pair inward. A single mismatch is enough to reject the string.

```
def is_palindrome(text):
    left = 0
    right = len(text) - 1

    while left < right:
        if text[left] != text[right]:
            return False
        left += 1
        right -= 1

    return True
```

Read the code

left < right keeps the indexes apart: a middle character needs no comparison with itself. **text[left]** reads the character at that index. A mismatch returns False before either pointer moves. After a match, both move inward. The final True is outside the loop, so it runs only after every required comparison passed.

Input / step	Compared characters	Result
madam: indexes 0 and 4	m and m	Match; move inward
madam: indexes 1 and 3	a and a	Match; move inward
madam: indexes 2 and 2	No comparison needed	Stop; True
abca: second comparison	b and c	Mismatch; False

```
print(is_palindrome("madam")) # True
print(is_palindrome("abca")) # False
print(is_palindrome(""))     # True
```

Scope: this function compares characters exactly. Capitalization, spaces, and punctuation matter. It does not yet perform the cleanup required by some interview variants.

Why O(n)? At most about $n/2$ pairs are checked; Big-O ignores the constant $1/2$. Two index variables use $O(1)$ extra space.

2. Pair sum in a sorted list

For [1, 2, 4, 6, 8] and target 10, begin with 1 + 8 = 9. It is too small, so move left to the larger value 2. Now 2 + 8 = 10. Sorting is what makes each movement safe.

```
def two_sum_sorted(nums, target):
    left = 0
    right = len(nums) - 1

    while left < right:
        total = nums[left] + nums[right]
        if total == target:
            return [left, right]
        elif total < target:
            left += 1
        else:
            right -= 1

    return []
```

Why these movement rules work

If the sum is too small, even the largest remaining partner cannot make the current left value work. Discard that left position. If the sum is too large, even the smallest remaining partner is too big for the current right value. Discard that right position.

elif means "otherwise, if". **else** handles the remaining case, where total exceeds target. `[]` is an empty list, returned when no pair exists. The condition `left < right` prevents reusing the same element.

```
print(two_sum_sorted([1, 2, 4, 6, 8], 10))
# [1, 4]: zero-based indexes of 2 and 8
```

LeetCode adaptation: one-based answers

Our teaching function returns Python indexes starting at 0. [LeetCode 167](#) asks for positions starting at 1. In its provided method, change the matching return to:

```
return [left + 1, right + 1]
```

Cost: each iteration shrinks the search area by one or more positions: $O(n)$ time, $O(1)$ space. An unsorted input does not support this reasoning; sorting also adds cost and may change original indexes.

3. Reverse a list in place

Swap the outside pair, move inward, and repeat. "In place" means modifying the existing list. We do not build a second list.

```
def reverse_list(nums):  
    left = 0  
    right = len(nums) - 1  
  
    while left < right:  
        nums[left], nums[right] = nums[right], nums[left]  
        left += 1  
        right -= 1
```

Understand the swap

Python evaluates both expressions on the right before assigning either destination on the left. This preserves both original values during the swap. Written with an explicit temporary variable, the same operation is:

```
temporary = nums[left]  
nums[left] = nums[right]  
nums[right] = temporary
```

Step	List
Start	[10, 20, 30, 40]
Swap indexes 0 and 3	[40, 20, 30, 10]
Swap indexes 1 and 2	[40, 30, 20, 10]
Pointers cross	Stop

```
nums = [10, 20, 30, 40]  
reverse_list(nums)  
print(nums) # [40, 30, 20, 10]
```

Return versus mutation: this function has no return statement, so it returns None. The result is in nums. Avoid writing `nums = reverse_list(nums)`, because that replaces your variable with None.

With [1, 2, 3, 4, 5], the first swap gives [5, 2, 3, 4, 1]. After the second swap, the center stays unchanged. Empty and one-element lists need no swaps.

Cost: O(n) time, O(1) extra space. Using `nums[::-1]` would instead create a new reversed list and use O(n) space.

4. Move Zeroes: read and write

Move nonzero values to the front in their original order. Then fill the unused tail with zeros. `read` visits every element; `write` marks the next available destination.

```
def move_zeroes(nums):
    write = 0

    for read in range(len(nums)):
        if nums[read] != 0:
            nums[write] = nums[read]
            write += 1

    for i in range(write, len(nums)):
        nums[i] = 0
```

`range(len(nums))` generates indexes 0 through `n - 1`. **`range(write, len(nums))`** starts at `write` and stops before `n`. Only kept values advance `write`; zeros advance `read` but not `write`.

Read value	Action	List afterward
0	Skip	[0, 1, 0, 3, 12]
1	Copy to index 0	[1, 1, 0, 3, 12]
0	Skip	[1, 1, 0, 3, 12]
3	Copy to index 1	[1, 3, 0, 3, 12]
12	Copy to index 2	[1, 3, 12, 3, 12]
End: write = 3	Zero indexes 3 and 4	[1, 3, 12, 0, 0]

The temporary duplicate values are expected. Only the first write positions are finished during the first loop. Because `write` never exceeds `read`, copying cannot damage values we have not read yet.

```
nums = [0, 5, 0, 7]
move_zeroes(nums)
print(nums) # [5, 7, 0, 0]
```

Checkpoint answer: after the first loop for this input, `write` equals 2: we kept 5 and 7. All-zero input becomes all zeros; input without zeros keeps its order.

Cost: two sequential loops add up to $O(n)$ work. They are not nested. Extra space is $O(1)$, and the function returns `None` because it modifies `nums` directly.

5. Remove sorted duplicates

In a sorted list, equal values are adjacent. Keep the first value, then copy a number only when it differs from the last value kept. Return k, the length of the finished prefix.

```
def remove_duplicates(nums):
    if not nums:
        return 0

    write = 1
    for read in range(1, len(nums)):
        if nums[read] != nums[write - 1]:
            nums[write] = nums[read]
            write += 1

    return write
```

if not nums is true for an empty list. For a nonempty list, index 0 is already kept, so write starts at 1. **nums[write - 1]** is the last unique value kept. The loop starts reading at index 1.

Input: [1, 1, 2, 2, 3]	Action	write afterward
read = 1; value = 1	Same as last kept: skip	1
read = 2; value = 2	Copy to index 1	2
read = 3; value = 2	Same as last kept: skip	2
read = 4; value = 3	Copy to index 2	3

```
nums = [2, 2, 2, 5, 5, 8]
k = remove_duplicates(nums)
print(k)          # 3
print(nums[:k])   # [2, 5, 8]
```

nums[:k] creates a slice from index 0 up to, but excluding, k. The function itself does not shorten the list. Values after index k - 1 are irrelevant and need not be cleared.

Common mistake: this method is for sorted input. In [2, 5, 2], the second 2 is separated from the first, so this comparison would keep it again.

Cost: O(n) time and O(1) auxiliary space. The slice in the printing example creates a separate list; it is not part of the algorithm.

6. Build a linked list

A node stores data and a reference to another node. head refers to the first node. Following next connections eventually reaches None in a list without a cycle.

```
class Node:
    def __init__(self, value):
        self.value = value
        self.next = None

first = Node(10)
second = Node(20)
third = Node(30)
first.next = second
second.next = third
head = first
```

Every new piece of syntax

class Node: defines an object type. `__init__` initializes a new instance when `Node(10)` is called. **self** is that particular instance. **self.value** and **self.next** are attributes stored on it. A dot accesses an attribute. **None** represents the absence of a node.

```
print(head.value)           # 10
print(head.next.value)     # 20
print(head.next.next.value) # 30
print(third.next)          # None
```

Read `head.next.value` as: start at head, follow one connection, then read value. `first.next = second` stores a reference to the second node, not the integer 20. `head = first` does not copy anything: both variables refer to the same node.

How this differs from a Python list

A Python list supports direct indexing, such as `nums[2]`. Our linked list has no index operation: to reach the third node, follow next twice. Reaching the kth node takes $O(k)$ time. Once you already have a node, reading its next reference takes $O(1)$.

LeetCode naming: its supplied `ListNode` usually stores the number as **.val**, while our teaching class uses **.value**. Use **.val** when reading data there. The middle and cycle algorithms below use only **.next**, so their logic works with either class.

7. Traverse and count nodes

Traversal means visiting nodes in order. These functions assume the list ends at None. If there is a cycle, ordinary traversal never finishes.

```
def print_list(head):
    current = head
    while current is not None:
        print(current.value)
        current = current.next

def count_nodes(head):
    current = head
    count = 0
    while current is not None:
        count += 1
        current = current.next
    return count
```

Following references without changing connections

current = head gives us a working reference. **current is not None** checks that a node exists before we read its attributes. **current = current.next** moves the reference forward; it does not change any node or connection. In contrast, assigning `current.next` would change a connection.

current before iteration	Printed / counted	current afterward
Node containing 10	10 / count becomes 1	Node containing 20
Node containing 20	20 / count becomes 2	Node containing 30
Node containing 30	30 / count becomes 3	None
None	Loop condition is false	Stop

```
print_list(head)          # Prints 10, 20, 30 on separate lines
print(count_nodes(head))  # 3
print(count_nodes(None))  # 0
```

Common mistake: forgetting `current = current.next` leaves `current` on the same node forever. Empty input is safe: the loop never begins, so `count` remains 0.

Cost: each of n nodes is visited once: $O(n)$ time. A reference and a counter take $O(1)$ auxiliary space.

8. Find the middle node

slow advances one connection per loop; fast advances two. When fast reaches the end, slow has covered about half the list. Return the node itself.

```
def middle_node(head):
    slow = head
    fast = head

    while fast is not None and fast.next is not None:
        slow = slow.next
        fast = fast.next.next

    return slow
```

Why the condition has two checks

and requires both conditions to hold. Python evaluates left to right and skips the second condition if the first is false. This prevents reading `.next` from `None`. When `fast` and `fast.next` exist, following two links is safe; the resulting reference may be `None`.

For 10 -> 20 -> 30 -> 40 -> 50	slow	fast
Start	10	10
After iteration 1	20	30
After iteration 2	30	50
fast.next is None: stop	Return node 30	50

For six nodes, fast moves from 10 to 30 to 50 to `None`, while slow moves from 10 to 20 to 30 to 40. This implementation therefore selects the **second middle** when the length is even.

```
middle = middle_node(head) # head is our 10 -> 20 -> 30 list
if middle is not None:
    print(middle.value)     # 20
```

Returning `slow.value` would return a number, not the required node. Returning a node retains its connection to the rest of the list. For empty input this teaching function returns `None`; for one node it returns that node.

Cost: $O(n)$ time, $O(1)$ extra space. Assume no cycle; otherwise fast cannot reach the end.

9. Detect a cycle

A cycle exists when following next eventually returns to an already visited node. Example: 10 -> 20 -> 30 -> 40, with 40.next pointing back to the same node 20.

```
def has_cycle(head):
    slow = head
    fast = head

    while fast is not None and fast.next is not None:
        slow = slow.next
        fast = fast.next.next
        if slow is fast:
            return True

    return False
```

slow is fast asks whether both references identify the exact same object. Comparing values is insufficient: two different nodes may contain equal values. Check for a meeting after moving, because both references initially start at head even in a list without a cycle.

Cycle: 40 connects back to 20	slow	fast
Start	10	10
After iteration 1	20	30
After iteration 2	30	20
After iteration 3	40	40: meeting

Why a meeting must happen

Once both references are inside the cycle, fast gains one node per loop relative to slow. The gap around the cycle eventually becomes zero. Without a cycle, fast reaches None and the function returns False. With n distinct reachable nodes, detection takes $O(n)$ time and $O(1)$ extra space.

```
a = Node(10)
b = Node(20)
c = Node(30)
a.next = b
b.next = c
print(has_cycle(a)) # False
c.next = b          # Connect back to the existing node b
print(has_cycle(a)) # True
```

A one-node self-loop also returns True. An empty list returns False. Do not run `print_list` on the cyclic example: it would not terminate.

Practice, review, and next steps

Use Python 3 on LeetCode. The five linked titles below are official problem pages. Warm up with the array exercises, then do sorted pair sum and linked-list exercises. The palindrome and reversal examples remain useful local drills.

[283. Move Zeroes - Easy](#)

Modify the input; preserve nonzero order. Write the read/write solution from memory.

[26. Remove Duplicates from Sorted Array - Easy](#)

Return k and leave the unique values in the first k positions.

[167. Two Sum II - Medium](#)

Use sorted order; return one-based indexes, unlike our teaching function.

[876. Middle of the Linked List - Easy](#)

Return the node itself; choose the second middle for even lengths.

[141. Linked List Cycle - Easy](#)

Compare node identity. The displayed pos describes the test setup; it is not an input parameter.

Use the supplied **class Solution** method and keep its name and parameters. Copy the algorithm body into it with the proper indentation. The teaching functions here use standalone snake_case names for clarity.

Explain a solution in an interview

State the assumptions (sorted? empty? cycle possible?). Describe what each pointer represents. Explain why moving it is safe. Trace one example, code the solution, then test a boundary case. Finish with time and space complexity.

Revision checklist

- [] I can explain index versus value versus node reference.
- [] I can choose inward, read/write, or slow/fast pointers.
- [] I understand mutation versus returning a new value.
- [] I can explain the None checks and node identity.
- [] I can trace empty, one-element, and even-length cases.

Checkpoint answers: abca mismatches at b/c; pair sum 10 with target 8 moves right; first reversal swap gives [5, 2, 3, 4, 1]; Move Zeroes keeps 2 values; duplicates return 3 and [2, 5, 8]; second.next.value is 30; the six-node middle is 40.

Day 2 tutorial complete. Revisit any practice you cannot yet explain. Your next PDF will be created after the next tutorial is complete.